GMXBUILDER User Manual

Document version	V1.0.4
Software	GMXBUILDER v0.9.19 or later
Author	Haochen Yang
Release date	2026-08-17
Status	Public release

This manual describes validated preparation of starting systems. Production simulation still requires minimization, sufficient equilibration, repeat sampling, and scientific review.

Contents

Select a section title or page number to follow the document link.

Change log

Document version	Date	Change	Author
V1.0.4	2026-08-17	Documented the managed GROMACS and GAFF2 runtimes, corrected execution-hardware, ion-placement, and explicit-count membrane-preview contracts, and synchronized installation, Web, CLI, API, and troubleshooting guidance with GMXBUILDER v0.9.19	Haochen Yang
V1.0.3	2026-08-15	Updated the verified V3 lipid assets, automatic external-asset bootstrap, installation contract, public documentation boundary, and current Martini 3 workflows	Haochen Yang
V1.0.2	2026-08-14	Documented strict prebuilt-lipid asset validation, current lipid-library status queries, automatic Martini 3 membrane orientation and box sizing, and corrected coarse-grained task navigation	Haochen Yang
V1.0.1	2026-08-11	Updated task routes, Task ID copy and recovery, Martini 3, DNA/RNA, CLI/API usage, output layout, and deployment boundaries	Haochen Yang
V1.0.0	2026-07-26	Initial release	Haochen Yang

1. Introduction

GMXBUILDER prepares GROMACS input packages for atomistic membrane-protein, pure-bilayer, and aqueous systems, and for supported Martini 3 coarse-grained systems. The Web, CLI, and HTTP API use the same validation rules.

1.1 Available workflows

Workflow	Purpose
Bilayer Builder	Process and orient a membrane protein, build a bilayer, add water and ions
Pure Bilayer System	Build a protein-free dry or solvated bilayer
Solvator	Build aqueous protein, canonical linear DNA/RNA, protein nucleic-acid, and compatible ligand systems
Martini 3 Bilayer Builder	Build a flat membrane or protein membrane CG system with explicit leaflet counts/compositions
Martini 3 Solvent Builder	Build a standard-protein aqueous CG system

Cards that are disabled in the installed interface are not current product capabilities. Query the application instead of relying on a static molecule count.

1.2 Check, Viewer, and Build

Every Check validates the current step and saves one authoritative task checkpoint. The Viewer reloads the saved coordinates. Later steps consume that checkpoint, so a changed upstream setting invalidates downstream results.

Final Build does not reconstruct the membrane, water, or ions. It assigns the topology and packages the last confirmed coordinates with the selected MDP stages and launcher script.

1.3 Scientific responsibility

A successful build establishes structural and topology consistency under the implemented checks. It does not establish a correct biological orientation, protonation state, ligand chemistry, equilibrium phase, or converged production trajectory. Review the scientific limitations before simulation.

2. Installation and service startup

2.1 Bootstrap requirements

Linux on x86-64 or AArch64 and Python 3.10 or later.

Git, CMake, a C++17 compiler, and Python's venv module.

Internet access during first installation.

The NVIDIA CUDA toolkit, including nvcc, only when the managed GROMACS runtime should be built with CUDA acceleration.

The installer manages the required GROMACS runtime, Python environment, GAFF2/AM1-BCC tools, force-field data, and verified prebuilt lipid assets. Users therefore do not need to install GROMACS, AmberTools, ACPYPE, Open Babel, Martini 3 data, or Python packages separately. Administrative access is not required when the bootstrap tools above are already present.

2.2 Unattended local installation

```
git clone https://github.com/AkaseToshiyuki/GMX-BUILDER.git
cd GMX-BUILDER
./install-local.sh
```

With no arguments, the installer is unattended: it binds to loopback on port 7788, assigns half of the detected CPU cores, derives a compatible queue size, and starts the user service. It reuses an explicitly selected or PATH-visible GROMACS 2026.0-or-newer executable. Otherwise it downloads the official GROMACS 2026.3 source archive, verifies its pinned SHA-256 digest, and builds a private runtime under the user's GMXBUILDER data directory. CUDA is enabled when nvcc is available; otherwise a complete CPU runtime is built.

The same installation creates a private GAFF2/AM1-BCC environment containing pinned AmberTools, ACPYPE, and Open Babel packages from conda-forge. It also downloads every separately distributed force-field port listed in scripts/external_assets.json and the V3 lipid archive from manifest-pinned HTTPS locations, verifies their digests, installs the locked Python environment, and populates the user cache. Git LFS, a GitHub token, and root access are not required. Run ./install-local.sh --help for command-line overrides or ./install-local.sh --interactive for prompts. One Task remains strictly serial; safe computation inside its current step may use multiple threads.

Select a compatible existing executable explicitly when desired:

```
./install-local.sh --gmx-bin /opt/gromacs/bin/gmx
```

Force a CPU-only managed GROMACS build even when CUDA is installed:

```
GMXBUILDER_GROMACS_FORCE_CPU=1 ./install-local.sh
```

2.3 Manual installation

```
python3 scripts/install_gromacs.py
python3 scripts/install_gaff_runtime.py
python3 scripts/install_external_assets.py
python3 scripts/fetch_prebuilt_assets.py
uv sync --frozen --no-dev
source .venv/bin/activate

gmxbuilder --version
gmxbuilder prebuilt-assets status
gmxbuilder prebuilt-assets install
```

Add the managed GROMACS executable to the current shell when using the manual sequence, or export its absolute path as GMX_BIN:

```
export GMX_BIN="$HOME/.local/share/gmxbuilder/runtime/gromacs-2026.3/bin/gmx"
export GMXBUILDER_GAFF_ENV="$HOME/.local/share/gmxbuilder/gaff-env"
```

The download bootstrap and asset installer verify the archive checksum, strict-library schema, and every included conformer-library entry before it writes the cache. A release contains only the force-field/lipid combinations that passed the production quality gates when it was built; other compatible combinations remain visibly unavailable. Installation does not overwrite a newer existing cache.

2.4 Start and inspect the service

```
gmxbuilder serve
```

Open <http://127.0.0.1:7788/>. Resource limits can be selected explicitly:

```
gmxbuilder serve \
  --host 127.0.0.1 --port 7788 \
  --cpu-cores 24 --task-threads 8 \
  --max-builds 3 --gpu-count 1
```

--task-threads must divide --cpu-cores. Set CUDA_VISIBLE_DEVICES before startup to expose a selected GPU subset or order. Use --gpu-count 0 for a CPU-only service.

Useful checks:

```
curl -fsS http://127.0.0.1:7788/health
systemctl --user status gmxbuilder.service
```

3. Web interface

3.1 Task ID and browser routes

After upload or task creation, save the displayed Task ID using the copy button. Browser routes contain only the workflow and step, for example:

```
/BilayerBuilder/Step3
/Solvator/Step2
```

The Task ID is deliberately not placed in the URL. Refreshing returns to the home page. Entering a saved Task ID resumes at the first incomplete visible step, or at Build when a completed package can be downloaded again.

Tasks expire according to the server retention policy. Task-private uploads, including custom lipid parameters, expire with the Task.

3.2 Bilayer Builder

Step 1 Input Structure

Upload PDB, mmCIF, or a supported gzip-compressed form. Review chains, retained small molecules, alternate locations, missing heavy atoms, modified residues, and warnings. Deselecting or renaming a component changes the saved input and requires another Check.

Step 2 Force Field

Select one coherent protein, membrane, ligand, and water-model combination. The UI disables incompatible combinations and the server repeats the check. Amber membranes use one complete Lipid21 or GAFF2 backend; CHARMM systems use compatible CHARMM lipids and CGenFF/CHARMM ligand parameters.

For GAFF2 ligands, confirm the integer net charge for the intended protonation state. For CHARMM ligands without an exact bundled template, upload the matching CGenFF MOL2 and STR files.

Step 3 Structure Processing

Compute protonation assignments at the selected pH, then review termini, modifications, and crosslinks. PROPKA is a static-structure suggestion, not constant-pH MD. Only force-field-specific atom-complete modifications can be selected. Unsupported chemistry remains visible and blocks export.

Step 4 Protein Orientation

Automatic orientation is an aid, not a biological annotation. Review the grey membrane-interface planes and all extracellular, intracellular, and transmembrane regions. Manual adjustments are saved by Check and used by the next step.

Step 5 Membrane Builder

Choose upper and lower leaflet compositions and lipid counts. Every lipid must be supported by the selected backend and have a valid conformer library entry. Before Check, the count preview follows the same round/trim/dominant-component padding rule as construction, and the estimated XY area uses the larger composition-weighted APL for an asymmetric bilayer. This is a construction estimate rather than an equilibrium APL; after Check, the saved checkpoint and its Viewer are authoritative. Review leaflet orientation, headgroups facing solvent, tails facing the bilayer core, packing around the protein, and the quality report.

Step 6 Solvent & Box

Z padding describes the requested water thickness outside the membrane interfaces; XY is inherited from the membrane. Check the box, water layers, atom counts, and that the membrane does not cross a periodic boundary.

Step 7 Ions and complete-system confirmation

Choose ion species, concentration, neutralization, exclusion radius, and placement method. Seeded uniform random replacement is the validated and recommended default and replaces complete waters at their oxygen coordinates. The formal-charge-ranked and dimensionless Metropolis site-optimization modes also replace complete waters, but are explicitly experimental heuristics; they are not equilibrium ion sampling and must not be interpreted as a predicted ion atmosphere. Check the total system, inspect water and ion distributions, then confirm the exact coordinates below the Viewer before continuing.

Step 8 Simulation Parameters and Build

Enable only the equilibration and production stages you want. Each stage owns its temperature, coupling, cutoffs, restraints, output intervals, COM removal, and duration. Hardware settings affect run_md.sh, not MDP physics. Build packages the confirmed ion checkpoint and does not rerun coordinate generation.

3.3 Pure Bilayer System

This workflow starts with force-field and lipid selection. It uses an independent implementation of membrane, solvation, ions, and export. Clear solvation to export a dry, relaxed bilayer; the dry package intentionally omits water, ions, MDP files, and run_md.sh.

3.4 Solvator

Solvator omits membrane construction and orientation. Box padding is measured from the full retained solute in all directions and pressure coupling defaults to isotropic.

Canonical linear DNA/RNA is displayed as polymer chains, not small molecules. Select CHARMM36m. Structure Processing uses native GROMACS/CHARMM36 data to construct hydrogens, 5 /3 termini, polymer bonds, and exact charge. Modified, circular, or broken nucleic-acid chains are blocked. This operation replaces uploaded nucleic-acid coordinates with the hydrogen-complete pdb2gmx output. Review every nucleic-acid chain in the Step 3 viewer before continuing.

3.5 Martini 3 Builders

Martini 3 uses two independent entries: Martini 3 Bilayer Builder and Martini 3 Solvent Builder. There is no environment selector inside either workflow. The bilayer workflow accepts an exact lipid count per leaflet, symmetric/asymmetric compositions, optional standard protein mapping, an independent PPM-like membrane-orientation Check, regular W water and ions. The installed official PC/PE/PG/PS/SM/sterol topology set is filtered to molecules that the pinned geometry builder can construct and the structural gates can identify. The solvent workflow requires a standard protein. Ligands, PTMs, glycans, nucleic acids, custom CG molecules, curved surfaces, and backmapping are not silently discarded; they are unavailable.

The periodic box is derived automatically from the positioned CG envelope, solvent padding and, for bilayers, a conservative construction area plus the explicit leaflet count. The construction area is an initial packing value, not an equilibrium APL; NPT equilibration relaxes the periodic area. Simulation Parameters exposes separate minimization, optional NVT, optional NPT, production, output/COM-removal, and execution-hardware controls; stages remain strictly serial and retain Martini 3-compatible non-bonded defaults.

4. Command-line interface

4.1 Discover commands and capabilities

```
gmxbuilder --help
gmxbuilder build --help
gmxbuilder serve --help
gmxbuilder martini3-bilayer --help
gmxbuilder martini3-solvent --help
gmxbuilder info --pdb protein.pdb
gmxbuilder list-ff
gmxbuilder list-water
gmxbuilder list-lipids
gmxbuilder lipid-library status
```

4.2 YAML atomistic build

```
system_name: membrane_system
output_dir: ./output
seed: 42

modules:
  input:
    pdb: ./protein.pdb
  forcefield:
    name: amber14sb
    lipid_ff: lipid21
    ligand_ff: none
    water_model: tip3p
  structure:
    pH: 7.0
    prepare_standard_termini: true
  orient:
    method: ppm
  membrane:
    lipid_type: POPC
    box_padding: 2.0
  solvation:
    box_padding: 2.0
  ions:
    cation: NA
    anion: CL
    concentration: 0.15
    neutralize: true
    ion_method: random
  topology: {}
  simparams:
    schema_version: 2
  export:
    write_mdp: true
    execution_hardware:
      mode: thread-mpi
      cpu_threads: 8
      mpi_ranks: 2
      use_gpu: true
      gpu_count: 1
      gpu_ids: [0]
      gmx_command: gmx
```

```
gmxbuilder build --config build.yaml
gmxbuilder build --config build.yaml --output ./another-output
```

Every selected lipid and retained molecule must be valid for the complete force-field combination. CLI failures use non-zero exit status and do not silently ignore unknown keys.

For YAML/CLI builds, execution hardware belongs to export.execution_hardware, not simparams. It changes the generated launcher and never changes the accepted molecular coordinates or MDP physics. HTTP Build requests use the separate modules.execution object shown by the installed OpenAPI schema.

4.3 Martini 3 example

```
gmxbuilder martini3-bilayer \
  --upper POPC:3 --upper CHOL:1 \
  --lower POPE:1 --lower POPG:1 \
  --lipids-per-leaflet 150 --padding 2 --salt 0.15 \
  --threads 8 --mpi-ranks 1 --gpu-ids 0 \
  --output ./martini-system
```

Use --pdb protein.pdb for a protein bilayer system. For an aqueous system use gmxbuilder martini3-solvent --pdb protein.pdb ....

5. HTTP API

5.1 Discovery and error handling

When the service is running:

Swagger UI: <http://127.0.0.1:7788/docs>

OpenAPI schema: <http://127.0.0.1:7788/openapi.json>

Useful discovery endpoints:

```
GET /health
GET /api/hardware
GET /api/task-types
GET /api/options
```

Clients must check both the HTTP status and the JSON body. 4xx denotes invalid input, unavailable capability, missing prerequisite, or expired Task; 5xx denotes a server-side failure. Do not continue after an error response.

5.2 Minimal Solvator sequence

```
API=http://127.0.0.1:7788

curl -sS -X POST "$API/api/upload-pdb" \
  -F "file=@protein.pdb" \
  -F "task_type=solvator"

TASK_ID=<returned-task-id>

curl -sS -X POST "$API/api/step/$TASK_ID/input" \
  -H "Content-Type: application/json" -d '{"config":{}}'

curl -sS -X POST "$API/api/step/$TASK_ID/forcefield" \
  -H "Content-Type: application/json" \
  -d '{"config":{"name":"amber14sb","lipid_ff":"none","ligand_ff":"none","water_model":"tip3p"}}'

curl -sS -X POST "$API/api/step/$TASK_ID/structure" \
  -H "Content-Type: application/json" \
  -d '{"config":{"pH":7.0,"prepare_standard_termini":true}}'

curl -sS -X POST "$API/api/step/$TASK_ID/solvation" \
  -H "Content-Type: application/json" \
  -d '{"config":{"box_padding":2.0}}'

curl -sS -X POST "$API/api/step/$TASK_ID/ions" \
  -H "Content-Type: application/json" \
  -d
    '{"config":{"cation":"NA","anion":"CL","concentration":0.15,"neutralize":true,"ion_method":"random"}}'
```

Bilayer clients also execute orient and membrane before solvation. Inspect saved checkpoints with GET /api/steps/{task_id}.

5.3 Build, queue, and download

```
curl -sS -X POST "$API/api/build" \
  -H "Content-Type: application/json" \
  -d '{
    "task_id":"'"$TASK_ID"'",
    "task_type":"solvator",
    "system_name":"solution_system",
    "modules":{
      "simparams":{"schema_version":2},
      "export":{"write_mdp":true}
    }
  }'

curl -sS "$API/api/build/$TASK_ID/queue-status"
curl -fL "$API/api/task/$TASK_ID/download" -o result.zip
```

Queue responses include position and an estimated start time. Save the Task ID before leaving the page. GET /api/tasks is an administrator endpoint and requires X-Admin-Token.

6. Output package and execution

6.1 Typical atomistic package

```
input.gro
input.pdb                 # optional for very large systems
topol.top
index.ndx
run_md.sh
README.txt
forcefield.itp
ffbonded.itp
ffnonbonded.itp
topol_Protein_chain_A.itp    # only when present
<lipid-or-ligand>.itp        # only when present
<water-model>.itp
<ion-parameters>.itp
mdp/
  mini.mdp
  equili_<n>.mdp             # enabled stages only
  production.mdp
  production_<n>.mdp         # segmented production when requested
```

Exact filenames depend on the selected system. topol.top is the authoritative include list. Martini packages additionally use toppar/, manifest.json, and CITATIONS.json.

6.2 Run the generated package

```
unzip result.zip -d simulation
cd simulation
chmod +x run_md.sh
./run_md.sh
```

Read README.txt and inspect every grompp and mdrun message. Do not suppress warnings without understanding their physical and topology implications.

7. Troubleshooting

7.1 A lipid is unavailable

The selected backend lacks either an exact topology or a conformer that passed the quality gates. Use the alternative shown by the UI or select another validated composition.

7.2 Bilayer orientation or packing looks wrong

Do not continue. Recheck protein orientation, leaflet assignment, conformer identity, composition, and the Step 5 quality report.

7.3 Automatic protein orientation is implausible

Treat the automatic score as a starting point. Use manual adjustment, Check again, and verify that the subsequent Viewer matches the saved orientation.

7.4 Water layers appear unequal

For membranes, padding is measured from the membrane interfaces. A protruding protein can make the visible solvent shape asymmetric without redefining the requested membrane-relative padding. Check numerical box and interface values.

7.5 Ions cluster or water is missing from the Viewer

Re-run Ion Check, verify the selected replacement method and exclusion radius, and inspect the exact complete-system Viewer. Do not proceed if coordinates do not match the reported counts.

7.6 ZIP lacks MDP files or run_md.sh

A dry bilayer intentionally omits them. For a solvated system, ensure that at least one simulation stage and MDP export were enabled, then rebuild from the confirmed checkpoint.

7.7 A Task cannot be resumed

Check the 32-character ID and the server retention period. Task-private custom lipids cannot be moved to another Task.

7.8 Automatic installation cannot build GROMACS

Confirm that CMake, a C++17 compiler, Python venv, sufficient disk space, and network access are available. To bypass a local CUDA-toolchain problem, retry with GMXBUILDER_GROMACS_FORCE_CPU=1. To use an administrator-provided compatible build, pass its gmx executable with --gmx-bin. Do not point the installer to GROMACS older than 2026.0, because the bundled Amber ff14SB port requires the newer preprocessing behavior.

8. Getting help

```
gmxbuilder --help
gmxbuilder list-ff
gmxbuilder list-water
gmxbuilder list-lipids
gmxbuilder lipid-library status
curl -fsS http://127.0.0.1:7788/health
```

When reporting a problem, include the GMXBUILDER version, Task ID, workflow step, complete error text, and a screenshot or log excerpt that contains no sensitive structure data.

Appendix A. Deployment security

Run as a non-root service account and restrict task-directory permissions.

Keep a public deployment bound to loopback behind a trusted TLS reverse proxy and set GMXBUILDER_DEPLOYMENT_MODE=public.

Configure the exact HTTPS origin, trusted proxy CIDRs, and strong global Basic or Bearer authentication.

Configure a strong GMXBUILDER_ADMIN_TOKEN; never log Task IDs or tokens.

Retain both application and reverse-proxy rate limits.

Install locked dependencies with uv sync --frozen --no-dev and review third-party licenses before redistribution.